



YA2YOS · KERNEL ENGINEERING

Ya2yOS

内核设计与工程实践

Rust 宏内核实现总览

RISC-V64

LoongArch64

Linux ABI

参赛队员：饶晓杰

指导老师：杨磊



CONTENTS

目录

从系统定位出发，沿实现路径走向可复核成果

01 · 系统定位

Rust 宏内核、Linux ABI、RISC-V64 / LoongArch64；面向真实 Linux 用户态路径与工具链负载。

02 · 系统介绍

从系统架构图出发，依次介绍进程管理、多核调度、内存管理、信号机制、文件系统和设备驱动。

03 · 关键增量

多核运行、CFS 调度、StarryOS 网络移植、uaccess、文件锁、syscall 扩展与模块重构。

04 · 发展规划

扩展文件系统类型、提升文件 I/O、持续丰富网络模块，并完成开发板实机运行与验证。



CHAPTER 01

系统定位

Rust 宏内核 · Linux ABI · RISC-V64 / LoongArch64



项目定位：Ya2yOS 是什么？

面向真实 Linux 用户态负载的 Rust 宏内核

Rust

内核主体 · 类型安全 · 显式所有权



宏内核

task · mm · fs · net · signal 共享地址空间



Linux ABI

syscall · errno · fd · signal · mmap

我们要解决什么问题

面向 glibc / musl、BusyBox、LTP 子集和 Rust 工具链等真实用户态路径，逐步补齐 Linux ABI；当前结论绑定到具体架构、配置和已完成的定向回归。

系统边界

Ya2yOS 直接运行在 QEMU / 部分板级适配环境中，不是用户态模拟器；以 Linux 用户可见语义为兼容边界，但不宣称完整 Linux ABI。

目标平台与负载

以 RISC-V64 / LoongArch64 为目标架构，重点验证 fork / exec / mmap、ext4 和 SMP 共享地址空间等高频路径；完整实机与压力回归仍在推进。



设计亮点：把复杂性收敛到可解释的内核路径

不仅实现接口，更统一对象、状态和资源生命周期

01 · 对象与生命周期

Rust 的引用计数与显式所有权贯穿任务、地址空间、文件和 socket；Process 与 TaskControlBlock 分离资源和执行上下文。

02 · SMP 并发一致性

以 Running → Blocked → Ready → Running 表达主要任务状态转换；简化 CFS 和 Remote-TLB MailBox 共同处理多核路径。

03 · Linux ABI 路径

uaccess 对满足条件的短复制使用架构快路径，其余回退安全路径；COW、socket、文件锁和 syscall 按当前验证范围逐步接入。

04 · 架构适配边界

RISC-V64 与 LoongArch64 共享主要内核接口，trap、页表 / TLB、timer、IPI 和 VirtIO 由架构层适配；部分硬件中断路径仍在完善。

05 · 验证口径

“支持”绑定到指定架构、配置下的构建、启动或定向回归；不把局部接口接入表述为完整 Linux 语义或全量压力验收。

06 · 演进方向

在语义和锁序正确的基础上，继续推进 VFS、I/O、网络 and 实机验证；性能收益以可追溯的同配置对照为准。



CHAPTER 02

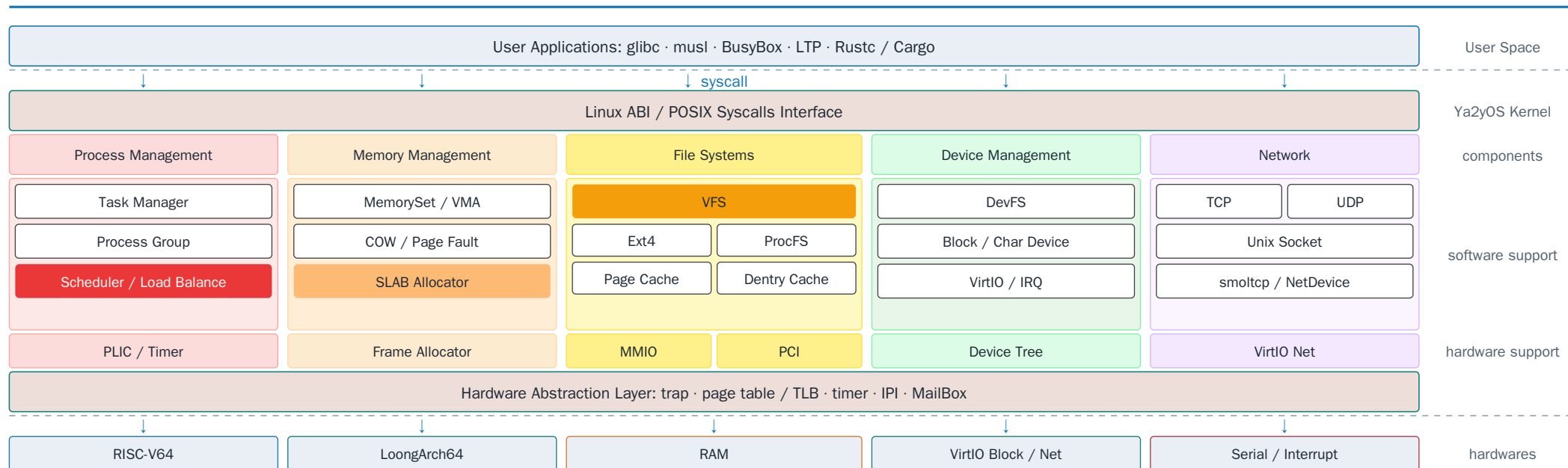
系统介绍

从系统架构图出发，沿进程、调度、内存、信号、文件系统与驱动走完整实现路径



系统架构

五类内核组件由 Linux ABI 统一入口衔接，并通过 HAL 收敛架构与设备差异





进程管理：对象、资源与生命周期

以 Process 和 TaskControlBlock 建立进程资源与线程执行边界

fork / clone

按 clone flags 创建或共享资源



PCB + TCB

进程资源 + 线程上下文



exec / exit / wait

替换、回收并向父进程报告

进程与线程分层

Process 持有地址空间、父子关系、信号与共享资源；TaskControlBlock 保存 TrapContext、调度状态和线程私有数据。

资源共享语义

clone flags 决定 MemorySet、FileTable、信号处理状态等资源是共享、复制还是新建；线程 affinity 保存在 TCB 而非进程全局。

主要生命周期路径

fork / clone 建立执行实体；execve 替换用户态映像；exit 与 waitpid 负责退出资源和状态的传递与回收。



多核调度：让可运行任务到达合适的 Hart

共享 ready queue 与线程 affinity 共同决定任务选择和空闲核唤醒



共享就绪队列

简化 CFS 使用共享 ready queue，并根据 vruntime 和权重选择任务；任务状态由原子字段或任务锁保护。

亲和性与唤醒

TCB 保存 Linux 可见的 CPU affinity mask；调度器保留暂不满足 affinity 的任务，空闲目标 Hart 可由平台唤醒。

当前边界

已实现 online-Hart mask、affinity 过滤和 idle-Hart 唤醒；运行中迁移、per-CPU queue、work stealing 和完整负载均衡仍属于后续演进。



内存管理：从 VMA 到跨核页表一致性

MemorySet 管理地址空间；缺页、COW 与 TLB shutdown 共同维护映射语义



地址空间对象

MemorySet 以 RwLock 包装 MemorySetInner；VMA、页表与映射生命周期集中管理，文件映射保留 backing inode。

缺页与 COW

file-backed 缺页先在地址空间锁外准备文件页，再在写锁阶段安装；匿名页和 COW 分别走按需分配与写保护复制路径。

跨核一致性

相关 PTE 更新遵循 UPDATE_LOCK → MemorySet 写锁顺序；remote TLB 通过专用 MailBox / IPI 收集 ACK，收敛前不释放旧帧。

用户访问边界

copy_from_user / copy_to_user 与页表检查共同保证 uaccess 失败返回 EFAULT，而不是越界访问内核内存。



信号机制：常规 handler 路径

pending、mask、signal frame 与 rt_sigreturn 组成已覆盖路径



投递与唤醒

信号写入目标线程或线程组的 pending 集；
若任务处于可中断阻塞状态，投递路径将其置为可运行，以便及时消费。

选择语义

pending 模块在 trap 返回前选择未被 mask 的信号；标准信号重复到达时保留 pending 位与首次 siginfo_t，不无限排队。

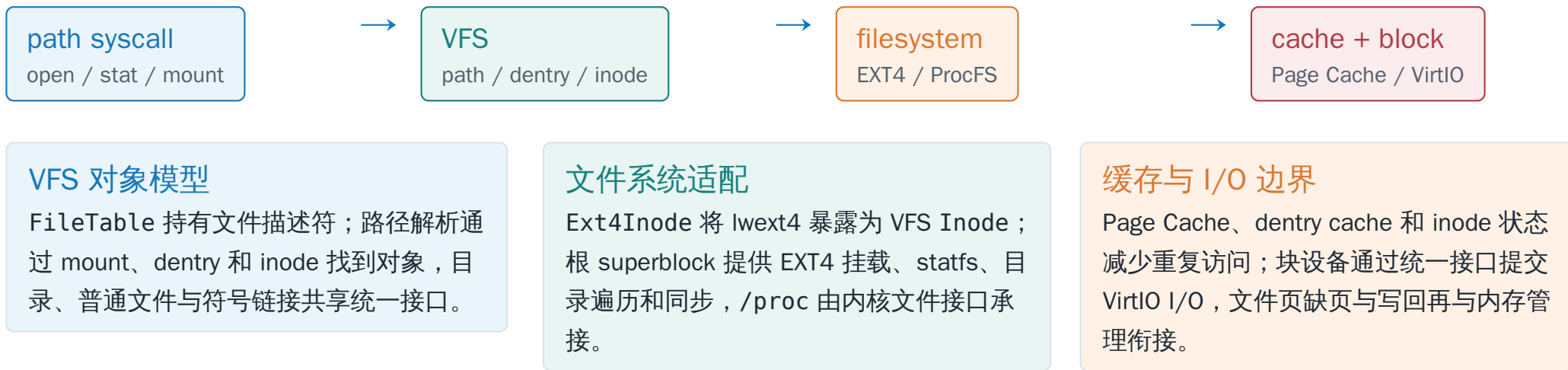
用户态 frame

有 handler 时在用户栈构造 signal frame；
rt_sigreturn 校验 frame magic 与布局后恢复寄存器、PC 和栈指针，非法 frame 返回 EINVAL。



文件系统：VFS 统一路径与 inode 语义

路径解析、缓存和 EXT4 / ProcFS 适配共同承接 Linux 文件 I/O





设备适配：把主要差异收敛到块设备与网卡接口

当前以 QEMU、同步块 I/O 和网络 poll 路径为主要验证范围



双架构适配

RISC-V64 主要使用 VirtIO-MMIO ;
LoongArch64 通过 PCI transport 适配
VirtIO。上层依赖 BlockDeviceImpl /
NetDeviceImpl , 但不同设备与 IRQ 完成
语义仍有边界。

块设备路径

块设备封装 VirtIO 队列并以同步访问为主 ;
文件系统经统一块接口提交读写 , 当前重
点是 QEMU 和已有回归路径。

网卡路径

VirtIO-net 维护 RX 描述符和 TX 缓冲 , 网络
服务主要通过 poll_interfaces() 推进 ; 完整
IRQ enable/disable/complete 与实板网络
验证属于后续工作。



CHAPTER 03

关键增量工作

并发运行 · Linux 语义 · 网络移植 · 工程结构



关键增量工作：三个方向

SMP 一致性、用户内存与 I/O 路径、ABI 工程化

01 · SMP 与一致性

简化 CFS、共享 ready queue、线程 affinity、idle-Hart 唤醒与 remote TLB MailBox / IPI / ACK。

02 · 用户内存与锁

uaccess 双路径、文件映射缺页、COW 以及 POSIX/OFD/BSD/lease 文件锁的主要接口与定向语义。

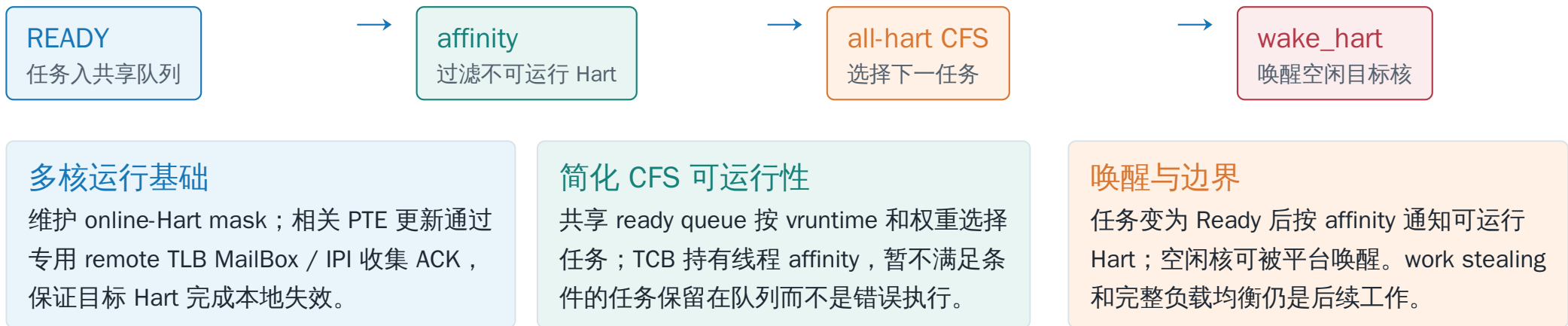
03 · 网络与 ABI 工程化

适配 StarryOS 网络抽象，接入 smoltcp 与 VirtIO-net；clone3、rseq、epoll 等入口按领域拆分并逐步验证。



简化 CFS 与多核一致性

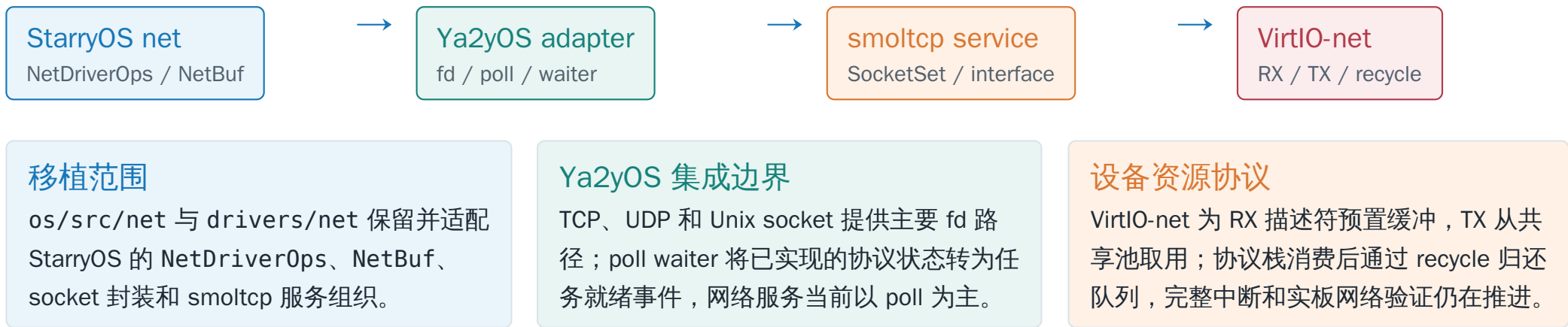
任务选择、线程 affinity、idle-Hart 唤醒与 remote TLB 分工明确





移植 StarryOS 网络模块并接入 Ya2yOS

保留 StarryOS 的网络抽象优势，在 Ya2yOS 的任务、fd 和双架构驱动边界中重新落地





uaccess 双路径与文件系统锁细化

用户指针访问与文件并发控制都必须同时满足常见路径效率和异常路径的 Linux 语义



uaccess 快路径边界

translate.rs 仅在当前活动地址空间、用户范围合法且长度满足限制时调用架构复制；其他场景回退逐页安全路径并传播 EFAULT。

安全与异常路径

跨页、延迟分配、文件页和 COW 回到通用映射路径；uaccess scope 与 trap fixup 处理内核态访问异常，最终向 syscall 传播 EFAULT。

细化用户可见文件锁

file_lock 提供 POSIX、OFD、BSD flock 与 lease 的主要接口；当前以基本语义、F_SETLKW 等定向等待路径和 close/exit 回收为主，底层锁表仍是简化实现。

保守的内部锁边界

lwext4 C API / bcache 的内部并发仍采用可等待的准入保护；先保证一致性与锁序，再逐步评估资源级细粒度并发。



丰富 syscall，并重构模块边界

扩大 ABI 覆盖面时，入口保持薄，核心语义与高风险状态机回到各自子系统



syscall 功能扩展

接入 clone3、rseq、epoll / poll 等入口，并按当前测试范围实现部分常用语义；key、inotify / fanotify、timerfd 等接口需按具体实现和验证结果区分。

按领域拆分入口

syscall 划分 task、mm、fs、net、sync、io_mpx、ipc、time、sys 等目录；handler 负责解码、复制、查表和 errno，语义不堆积在大分发函数。

提高内聚度

signal 拆为 delivery/pending/frame/timer；MemorySet 拆分 handle、pagefault、fork_clone 等；网络服务、驱动和文件锁也按资源职责拆分。



CHAPTER 04

发展规划

文件系统扩展 · I/O 优化 · 网络完善 · 开发板实机运行



发展规划：从已验证路径走向更完整能力

扩展 VFS、I/O、网络 and 实机验证；所有优化以可复核证据为前提

01 · 支持更多文件系统

在统一 VFS、dentry/inode 和 mount 语义下接入更多文件系统类型；完善不同文件系统的路径解析、权限、元数据和挂载参数兼容性。

02 · 提高文件系统 I/O 速率

围绕 PageCache 命中、顺序/批量读写、跨页复制和块设备提交路径减少重复工作；使用同配置、成功收尾的基准和文件系统回归验证优化收益。

03 · 继续丰富网络模块

补齐 socket 选项、协议语义、路由与设备事件处理；持续加强 TCP / UDP / Unix socket 与 poll / epoll、VirtIO-net 之间的一致性。

04 · 在开发板上成功运行

完成实机启动、内存与中断初始化、块设备/网卡驱动和串口观测；在开发板上跑通用户态程序、文件 I/O、网络通信与压力回归。



Ya2y0S

内核设计与工程实践

谢谢！